

TP6 — Application Web de Gestion des Produits

Spring MVC · Hibernate · MySQL · Authentification & Rôles

Réalisé par	Hasnae Elballaoui
Encadré par	M. Mohammed Echerraadi
Filière	Génie Logiciel & Systèmes d'Information
Année académique	2025 — 2026

1. Introduction

Ce rapport présente le Travail Pratique N°6 (TP6) réalisé dans le cadre du module Java EE. L'objectif est de concevoir et implémenter une application web de gestion de produits en utilisant le framework Spring MVC côté contrôleur, Hibernate comme couche ORM pour la persistance, et MySQL comme système de gestion de base de données relationnelle.

L'application offre un CRUD complet sur les produits (ajout, consultation, modification, suppression), une recherche par mot-clé ou identifiant, ainsi qu'un système d'authentification basé sur les rôles (ADMIN / USER) sécurisé par des filtres Servlet.

2. Architecture du Projet

2.1 Structure Maven

Le projet est packagé en fichier WAR (Web Application Archive) déployable sur Tomcat 9. La structure Maven standard adoptée est la suivante :

```
GestionDesProduitsSPRING/
├── pom.xml
├── src/main/
│   ├── java/
│   │   ├── controller/   ProduitController.java
│   │   ├── dao/          ProduitDAO, ProduitDAOImpl, UserDAO, UserDAOImpl
│   │   ├── entities/     Produit.java, User.java
│   │   ├── filters/      AuthFilter.java, RoleFilter.java
│   │   └── services/     ProduitService, ProduitServiceImpl, UserService,
│   │                       UserServiceImpl
│   ├── util/             HibernateUtil.java
│   ├── resources/
│   │   └── hibernate.cfg.xml
│   └── webapp/
│       ├── views/        index.jsp, login.jsp, error.jsp
│       ├── WEB-INF/      web.xml, application-servlet-config.xml, spring-beans.xml
│       └── index.jsp
```

2.2 Patron d'Architecture en Couches

L'application respecte strictement le patron MVC (Model-View-Controller) combiné à une architecture en couches :

Couche	Classes / Fichiers	Responsabilité
Présentation (View)	index.jsp, login.jsp, error.jsp	Interface utilisateur JSP avec JSTL
Contrôleur	ProduitController.java	Routing HTTP, orchestration des services
Service (Métier)	ProduitServiceImpl, UserServiceImpl	Validation métier, logique applicative
DAO (Persistance)	ProduitDAOImpl, UserDAOImpl	Accès BDD via Hibernate/HQL
Entités (Modèle)	Produit.java, User.java	Mapping JPA vers tables MySQL

Couche	Classes / Fichiers	Responsabilité
Filtres	AuthFilter, RoleFilter	Sécurité, authentification, contrôle des rôles

3. Dépendances et Technologies

Technologie	Version	Rôle
Spring MVC (spring-webmvc)	5.3.30	Framework contrôleur web
Spring ORM (spring-orm)	5.3.30	Intégration Spring-Hibernate
Hibernate Core	5.6.15.Final	ORM, mapping objet-relationnel
MySQL Connector/J	8.2.0	Driver JDBC MySQL
javax.servlet-api	4.0.1 (provided)	API Servlet pour Tomcat 9
JSTL	1.2	Bibliothèque de balises JSP
Java	11	Langage de développement
Apache Tomcat	9.x	Serveur d'application web
MySQL	8.x	SGBD relationnel

4. Entités JPA

4.1 Entité Produit

L'entité Produit est mappée sur la table produits en base de données. Elle comporte quatre attributs : un identifiant auto-généré, un nom, une description et un prix.

```
@Entity
@Table(name = "produits")
public class Produit {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_produit")
    private Long idProduit;

    @Column(name = "nom", nullable = false, length = 200)
    private String nom;

    @Column(name = "description", length = 500)
    private String description;

    @Column(name = "prix", nullable = false)
    private Double prix;
    // Constructeurs, Getters, Setters ...
}
```

4.2 Entité User

L'entité User représente un utilisateur de l'application. Le champ role prend les valeurs ADMIN ou USER, déterminant les droits d'accès.

```
@Entity
```

```
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idUser;

    @Column(nullable = false, unique = true, length = 100)
    private String username;

    @Column(nullable = false, length = 255)
    private String password;

    @Column(nullable = false, length = 20)
    private String role; // "ADMIN" ou "USER"
}
```

5. Couche DAO (Data Access Object)

5.1 Interface ProduitDAO

L'interface ProduitDAO définit le contrat de persistance pour l'entité Produit. Elle expose les six opérations fondamentales :

- addProduit(Produit p) — insertion d'un nouveau produit
- getProduitById(Long id) — récupération par clé primaire
- getAllProduits() — liste complète triée par ID
- updateProduit(Produit p) — mise à jour d'un produit existant
- deleteProduit(Long id) — suppression par identifiant
- rechercherParMotCle(String motCle) — recherche HQL LIKE sur nom et description

5.2 Implémentation ProduitDAOImpl

ProduitDAOImpl utilise le patron Session Hibernate manuel : chaque méthode ouvre une session, démarre une transaction si nécessaire, exécute l'opération et ferme la session dans le bloc finally. La recherche textuelle utilise une requête HQL paramétrée avec LOWER() et LIKE pour une recherche insensible à la casse :

```
// Requête HQL de recherche par mot-clé
String kw = "%" + motCle.toLowerCase() + "%";
Query<Produit> query = session.createQuery(
    "FROM Produit p WHERE LOWER(p.nom) LIKE :kw OR LOWER(p.description) LIKE :kw",
    Produit.class
);
query.setParameter("kw", kw);
return query.list();
```

5.3 Interface UserDAO et Implémentation

UserDAO expose trois méthodes : findByUsernameAndPassword() pour l'authentification (requête HQL unique), addUser() pour l'insertion et existsByUsername() pour vérifier l'unicité avant insertion. L'implémentation UserDAOImpl suit le même patron de session Hibernate.

6. Couche Service (Logique Métier)

6.1 ProduitService et ProduitServiceImpl

La couche service introduit la validation métier avant toute délégation au DAO. Les deux règles appliquées sont :

- Le nom du produit ne peut pas être vide ou uniquement composé d'espaces.
- Le prix doit être un nombre positif ou nul.

En cas de violation, une `IllegalArgumentException` est levée et remontée au contrôleur pour affichage à l'utilisateur. Le service est injecté par Spring via le fichier `spring-beans.xml` (injection par setter XML).

6.2 UserService et UserServiceImpl

`UserServiceImpl` gère l'authentification et l'initialisation des comptes par défaut. La méthode `initDefaultUsers()` est déclarée comme `init-method` dans `spring-beans.xml` et s'exécute au démarrage de l'application pour insérer les trois comptes par défaut (si absents) :

Utilisateur	Mot de passe	Rôle
admin	admin123	ADMIN
user	user123	USER
cherradi	tdia2024	ADMIN

7. Contrôleur Spring MVC — ProduitController

`ProduitController` est annoté `@Controller` et orchestre l'ensemble des interactions HTTP. Il utilise `@Autowired` pour recevoir les services `ProduitService` et `UserService` par injection de dépendances.

7.1 Mappings des routes

Méthode HTTP	URL	Handler	Description
GET	/login	loginForm()	Affiche le formulaire de connexion
POST	/login	loginSubmit()	Valide les identifiants, crée la session
GET	/logout	logout()	Invalide la session, redirige vers /login
GET	/produits	listeProduits()	Affiche la liste complète des produits
GET	/produits/search	searchProduit()	Recherche par ID ou mot-clé
POST	/produits/add	addProduit()	Ajoute un nouveau produit (ADMIN)
GET	/produits/edit	editProduit()	Charge le formulaire d'édition (ADMIN)

Méthode HTTP	URL	Handler	Description
POST	/produits/update	updateProduit()	Met à jour un produit (ADMIN)
GET	/produits/delete	deleteProduit()	Supprime un produit (ADMIN)

7.2 Patron PRG (Post/Redirect/Get)

Toutes les actions POST (add, update, delete) retournent un redirect:/produits après l'opération. Ce patron PRG (Post-Redirect-Get) évite les soumissions multiples lors d'un rechargement de page. Les messages de succès ou d'erreur sont transmis via RedirectAttributes qui les encodent dans l'URL de redirection.

8. Sécurité — Filtres Servlet

8.1 AuthFilter — Authentification

AuthFilter intercepte toutes les requêtes HTTP entrantes. Il laisse passer les URLs publiques (page de login, ressources statiques CSS/JS/images) et redirige vers /login les requêtes sans session active.

```
// URLs publiques exemptées du filtre
boolean estPublic = uri.equals(contextPath + "/login")
|| uri.endsWith(".css")
|| uri.endsWith(".js")
|| uri.endsWith(".png")
|| uri.endsWith(".ico");
```

8.2 RoleFilter — Contrôle des Rôles

RoleFilter est appliqué spécifiquement sur les URLs des opérations d'écriture (/produits/add, /produits/edit, /produits/update, /produits/delete). Il vérifie que l'utilisateur connecté possède le rôle ADMIN ; dans le cas contraire, il redirige avec un message d'erreur.

```
// Vérification du rôle ADMIN
User user = (User) session.getAttribute("userConnecte");
if (user != null && "ADMIN".equals(user.getRole())) {
    chain.doFilter(request, response); // Autorisé
} else {
    resp.sendRedirect(req.getContextPath()
        + "/produits?messageErreur=Accès+refusé+:+rôle+ADMIN+requis");
}
```

9. Configuration

9.1 Hibernate — hibernate.cfg.xml

Le fichier hibernate.cfg.xml configure la connexion MySQL et le comportement d'Hibernate :

- Base de données : gestion_produits sur localhost:3306
- Dialect : MySQL8Dialect pour une compatibilité MySQL 8.x

- `hbm2ddl.auto = update` : les tables sont créées automatiquement si absentes, mises à jour sinon
- `show_sql = true` : les requêtes SQL générées sont affichées dans la console
- Pool de connexions : 10 connexions simultanées

9.2 Spring — spring-beans.xml

Le fichier `spring-beans.xml` déclare les beans Spring et configure l'injection de dépendances par XML. Il définit les DAO, les services (avec leurs injections via setter) et appelle `initDefaultUsers` comme méthode d'initialisation au démarrage du contexte Spring.

9.3 Dispatcher — application-servlet-config.xml

Ce fichier configure le `DispatcherServlet` de Spring MVC : activation de l'annotation `@Controller`, résolution des vues JSP (préfixe `/views/`, suffixe `.jsp`) et activation du scan des composants Spring dans les packages du projet.

10. Utilitaire Hibernate — HibernateUtil

`HibernateUtil` implémente le patron Singleton pour la `SessionFactory` Hibernate. La `SessionFactory` est créée une seule fois au chargement de la classe (bloc static) depuis le fichier `hibernate.cfg.xml`. Elle expose trois méthodes :

- `getSessionFactory()` — retourne la `SessionFactory` partagée
- `getSession()` — ouvre et retourne une nouvelle `Session` Hibernate
- `shutdown()` — ferme proprement la `SessionFactory` à l'arrêt

Ce singleton garantit qu'une seule `SessionFactory` coûteuse est créée, tandis que des sessions légères sont ouvertes pour chaque opération de persistance.

11. Vues JSP

11.1 login.jsp — Page d'authentification

La page `login.jsp` présente un formulaire HTML qui soumet les identifiants en POST vers `/login`. Elle affiche un message d'erreur si les identifiants sont incorrects et pré-remplit le champ `username` après une tentative échouée.

11.2 index.jsp — Interface principale

La vue principale `index.jsp` est polymorphe : elle affiche la liste des produits, le formulaire d'ajout (visible pour tous les utilisateurs authentifiés), le formulaire de modification (pré-rempli lors d'un clic Éditer) et les boutons d'action (Supprimer, Éditer) réservés aux administrateurs. La page utilise JSTL pour l'itération sur la liste et l'affichage conditionnel selon le rôle.

11.3 error.jsp — Page d'erreur

La page error.jsp gère les erreurs HTTP (404, 500) et affiche un message convivial avec un lien de retour vers la liste des produits.

12. Flux d'Exécution

12.1 Flux d'authentification

Le processus d'authentification se déroule en cinq étapes :

1. Utilisateur soumet login.jsp (POST /login)
2. ProduitController.loginSubmit() appelle UserService.authenticate()
3. UserServiceImpl délègue à UserDAO.findByUsernameAndPassword() (requête HQL)
4. Si trouvé → session.setAttribute("userConnecte", user) + redirect:/produits
5. Si non trouvé → retour login.jsp avec message d'erreur

12.2 Flux CRUD — Ajout d'un produit

1. ADMIN remplit le formulaire (POST /produits/add)
2. RoleFilter vérifie le rôle ADMIN → autorise
3. ProduitController.addProduit() valide les paramètres HTTP
4. ProduitService.addProduit() applique les règles métier (nom non vide, prix >= 0)
5. ProduitDAOImpl.addProduit() ouvre session Hibernate → session.save() → commit
6. Hibernate génère : INSERT INTO produits (nom, description, prix) VALUES (...)
7. redirect:/produits (PRG Pattern) avec messageSucces

13. Conclusion

Ce TP6 met en oeuvre une application web Java EE complète respectant les bonnes pratiques de développement logiciel : séparation stricte des couches (Controller, Service, DAO, Entity), patron de conception Singleton pour la SessionFactory Hibernate, patron PRG pour éviter les double-soumissions, sécurité par filtres Servlet avec contrôle fin des rôles, et injection de dépendances Spring par configuration XML.

L'architecture en couches garantit la maintenabilité du code : toute modification de la base de données n'impacte que la couche DAO, la logique métier reste isolée dans la couche service, et le contrôleur Spring MVC se concentre uniquement sur le routage HTTP et la gestion des vues.

Les compétences acquises lors de ce TP couvrent : la configuration d'un projet Maven multi-couches, l'utilisation d'Hibernate en mode Session manuelle, la configuration du DispatcherServlet Spring MVC, la sécurisation d'une application web par filtres Servlet, et l'application du patron MVC dans un contexte Java EE professionnel.